

Step-by-Step Project Walkthrough

OpenAlex DBMS Comparison: PostgreSQL 16 vs Neo4j 5

Students Hamza Abdul Kader, 2230150
Leonardo Marasca, 2217140
Purpose Detailed explanation for understanding, presenting, and defending the project
Dataset OpenAlex AI/RAG research paper subset
Systems PostgreSQL 16 and Neo4j 5

This document assumes the reader did not study all lecture slides and explains both the technical implementation and the conceptual reasons behind it.

Contents

1	One-Sentence Summary	4
2	How Leo Should Use This Document	4
3	What the Project Is About	4
4	Concepts Leo Needs Before the Project Details	4
4.1	The Core Difference: Joins vs Traversal	6
5	What We Chose and Why	6
6	The Full Project Pipeline	6
7	Repository Structure	7
8	Step 1: Collecting Data from OpenAlex	8
9	Step 2: Cleaning and Normalizing the Data	8
9.1	Important Cleaning Problems	9
9.2	Why We Did Not Load the Raw JSON Directly	9
9.3	Important ETL Decisions	10
9.4	What “Normalize” Means Here	10
10	Step 3: Dataset Size	10
11	Step 4: Running the Databases with Docker	11
12	Step 5: PostgreSQL Implementation	11
12.1	Why These Tables Exist	12
12.2	Primary Keys, Foreign Keys, and Indexes	12
12.3	How PostgreSQL Represents Relationships	12
12.4	Loading PostgreSQL	13
13	Step 6: Neo4j Implementation	13
13.1	Why This Graph Model Fits OpenAlex	13
13.2	Neo4j Labels, Properties, and Constraints	14
13.3	How Neo4j Represents Relationships	14
13.4	Loading Neo4j	14

14 Step 7: Validating the Load	15
15 Step 8: Writing Equivalent Queries	15
15.1 What the 11 Queries Test	15
15.2 What “Equivalent Query” Means	16
15.3 Example Query	16
16 Step 9: Benchmarking	17
16.1 Why We Use Warmups and Averages	17
16.2 Why PostgreSQL Uses EXPLAIN ANALYZE	18
16.3 Why Neo4j Uses PROFILE	18
16.4 What the Benchmark Does Not Prove	18
17 Step 10: Benchmark Results	19
17.1 Supplemental Graph-Focused Benchmark	19
17.2 How to Explain the Result Without Overclaiming	20
17.3 Why PostgreSQL Wins Many Queries Here	20
17.4 Why Neo4j Still Matters	20
17.5 What the Professor May Care About	21
18 Step 11: Visual Evidence	21
19 Step 12: Final Deliverables	21
20 How to Explain the Project in the Presentation	22
21 What Leo Should Know Before Presenting	22
22 How This Connects to the Lecture Slides	23
22.1 Lecture Concepts Explained Without the Slides	23
23 Possible Professor Questions and Short Answers	24
24 What We Should Do Now	25
24.1 Immediate Checklist	25
24.2 What Leo Should Focus on Learning	25
24.3 Suggested Presentation Split	26
24.4 Slide-by-Slide Speaking Notes	26
24.5 What Not to Do at the Last Minute	27

24.6 If the Professor Asks “What Did Leo Do?”	27
25 Commands to Reproduce the Project	28
26 Final Simple Conclusion	28

1 One-Sentence Summary

We built a project that collects AI-related research paper data from OpenAlex, loads the same cleaned dataset into PostgreSQL and Neo4j, runs equivalent SQL and Cypher queries, benchmarks them, and compares which database model is better for each type of question.

2 How Leo Should Use This Document

This file is meant to make the project understandable without needing to read every lecture slide first. It explains three things:

- **What we built:** the dataset, scripts, databases, queries, benchmarks, report, and presentation.
- **Why we built it this way:** the database concepts behind each choice.
- **What to do now:** how to prepare for the presentation and what points to be ready to explain.

Leo does not need to memorize every command. The important goal is to understand the story clearly enough to answer questions. If the professor asks about a file, a query, or a modeling decision, Leo should be able to explain the reasoning in simple words.

3 What the Project Is About

The project is a DBMS comparison. We compare:

- **PostgreSQL 16:** a relational database, based on tables and SQL;
- **Neo4j 5:** a graph database, based on nodes, relationships, and Cypher.

The dataset is about AI research papers from OpenAlex. This is a good dataset because research papers naturally have many relationships:

- authors write papers;
- papers have topics;
- papers cite other papers;
- authors can be connected through citations, shared papers, or shared topics.

This makes it suitable for comparing a relational model and a graph model.

4 Concepts Leo Needs Before the Project Details

This section explains the basic technical ideas used in the project. These are the concepts that matter most for understanding the work.

Concept	Meaning in simple words
DBMS	A Database Management System. It is software used to store, query, update, and manage data. PostgreSQL and Neo4j are both DBMSs, but they organize data differently.
Relational database	A database based on tables. Data is stored in rows and columns. PostgreSQL is relational.
Graph database	A database based on nodes and relationships. Neo4j is a graph database. It is useful when the relationships are as important as the entities.
Table	A structured collection of rows. For example, the papers table stores one row per paper.
Node	A graph object. In our Neo4j database, a paper is a Paper node, an author is an Author node, and a topic is a Topic node.
Relationship / edge	A connection between two nodes. For example, (Author)-[:AUTHORED]->(Paper) means an author wrote a paper.
Property	A value stored on a node or relationship. For example, a paper node has properties such as title, year, DOI, and citation count.
Primary key	A column or group of columns that uniquely identifies a row. For example, paper_id uniquely identifies a paper in PostgreSQL.
Foreign key	A column that points to a row in another table. It protects the consistency of relationships between tables.
Bridge table	A table used to represent a many-to-many relationship. For example, one paper can have many authors and one author can write many papers, so we use paper_authors .
Index	A data structure that helps the DBMS find rows or nodes faster. It is like an organized lookup path.
SQL	The query language used by PostgreSQL. It is very strong for filtering, joining, grouping, sorting, and aggregation.
Cypher	The query language used by Neo4j. It is strong for expressing graph patterns such as paths between authors, papers, and topics.
API	A way for programs to request data from another system. We use the OpenAlex API to download paper data.
JSON	A nested data format commonly returned by APIs. OpenAlex returns paper data as JSON objects.
JSONL	A file format where each line is one JSON object. We save raw OpenAlex works as JSONL because it is convenient for API records.
CSV	A simple table-like file format. We transform JSON into CSV because both PostgreSQL and Neo4j can import CSV easily.
ETL	Extract, Transform, Load. Extract data from OpenAlex, transform it into clean tables, then load it into PostgreSQL and Neo4j.
Benchmark	A controlled measurement of execution time. We run equivalent queries in both systems and compare average time.
Warmup run	A run that is not counted in the final result. It reduces the effect of first-time setup and cache behavior.

4.1 The Core Difference: Joins vs Traversal

The main conceptual difference is this:

- In PostgreSQL, relationships are reconstructed by joining tables.
- In Neo4j, relationships are stored directly as edges and traversed as paths.

Example:

Which authors are connected because one author's paper cites another author's paper?

In PostgreSQL, we need several joins between `paper_authors`, `citations`, and `authors`. In Neo4j, we write the relationship as a path:

```
(Author)-[:AUTHORED]->(Paper)-[:CITES]->(Paper)<-[:AUTHORED]-(Author)
```

This is the central reason our project is meaningful. The same real-world question can be represented in two different database models.

5 What We Chose and Why

At the beginning, we had to choose a project topic and a dataset. We selected OpenAlex because:

- it is open and public;
- it has an API;
- it contains scholarly works, authors, topics, and citations;
- it is related to AI and RAG, which makes the topic more interesting.

The official links used in the proposal were:

- OpenAlex website: <https://openalex.org/>
- OpenAlex data documentation: <https://help.openalex.org/hc/en-us/articles/24397285563671-About->

The professor approved the proposal, so we continued with this topic.

6 The Full Project Pipeline

The full pipeline is:

OpenAlex API

- > raw JSONL records
- > cleaned normalized CSV files
- > PostgreSQL relational tables
- > Neo4j graph nodes and relationships

-> equivalent SQL and Cypher queries
-> benchmark results
-> charts, report, and presentation

Every step has a reason:

- We use OpenAlex because it gives real academic data.
- We save raw JSONL so the original API records are preserved.
- We produce CSV files because both databases can load the same files.
- We load the same logical data into PostgreSQL and Neo4j so the comparison is fair.
- We write equivalent queries so each system answers the same question.
- We benchmark execution time to compare behavior, not only syntax.
- We create a report and slides so the project can be explained from requirements to conclusions.

The most important fairness principle is:

Both PostgreSQL and Neo4j must receive the same cleaned dataset. Otherwise, the comparison would not be valid.

7 Repository Structure

The repository is organized like this:

```
src/  
  collect_openalex.py  
  benchmark_queries.py  
  generate_benchmark_charts.py  
  
database/postgres/  
  schema.sql  
  load.sql  
  queries.sql  
  
database/neo4j/  
  constraints.cypher  
  import.cypher  
  queries.cypher  
  
benchmarks/  
  queries.json  
  results/benchmark_results.csv  
  
docs/  
  project_report.pdf  
  OpenAlex_DBMS_Comparison_Sapienza.pptx  
  figures/
```

The important idea is:

- `src/` contains Python scripts;
- `database/` contains database schema, load, and query files;
- `benchmarks/` contains query definitions and results;
- `docs/` contains the report, presentation, and screenshots.

8 Step 1: Collecting Data from OpenAlex

The data collection script is:

```
src/collect_openalex.py
```

We run it with:

```
python src/collect_openalex.py --per-term 75 --per-page 75
```

The script searches OpenAlex for AI-related terms:

- retrieval augmented generation;
- large language models;
- artificial intelligence;
- machine learning.

It saves the raw API result as:

```
data/raw/openalex_works.jsonl
```

JSONL means one JSON object per line. This is useful because OpenAlex returns nested JSON data.

9 Step 2: Cleaning and Normalizing the Data

OpenAlex data is nested. For example, a paper object can contain authors, topics, citations, abstract information, and metadata all inside one JSON object.

Databases are easier to load if the data is normalized into separate CSV files. So the script creates:

```
data/processed/papers.csv
data/processed/authors.csv
data/processed/topics.csv
data/processed/paper_authors.csv
data/processed/paper_topics.csv
data/processed/citations.csv
```

9.1 Important Cleaning Problems

We solved these problems:

- **Nested JSON:** converted into flat CSV files.
- **Abstract inverted index:** reconstructed abstracts into readable text.
- **Duplicate relationships:** removed duplicate paper-author pairs.
- **External citations:** kept only citations where both papers exist in our selected subset.

This step is important because it is the ETL part of the project:

Extract: OpenAlex API

Transform: clean and normalize JSON

Load: import CSV files into PostgreSQL and Neo4j

9.2 Why We Did Not Load the Raw JSON Directly

OpenAlex records are rich but messy for direct database comparison. A single work can contain nested authorships, nested topics, abstract information, IDs, URLs, citation references, and metadata. If we loaded this raw structure directly into only one database, the comparison would become unfair and hard to explain.

Instead, we normalized the information into a shared tabular format first. This gives us one clean intermediate dataset. Then both PostgreSQL and Neo4j are loaded from that same intermediate dataset.

This is important because it separates two concerns:

- **Data preparation:** done once in Python.
- **Database comparison:** done after both databases contain the same cleaned data.

9.3 Important ETL Decisions

Decision	Reason
Use a focused AI/RAG subset	The complete OpenAlex dataset is very large. A focused subset is easier to run locally and still contains useful paper-author-topic-citation relationships.
Keep raw JSONL	It preserves the original API response, so the transformation can be checked later.
Create normalized CSV files	CSV is easy to import into both PostgreSQL and Neo4j, which makes the comparison fairer.
Deduplicate entities	The same author, topic, or relationship can appear multiple times in API results. Duplicates would break primary keys or distort counts.
Keep only internal citations	Many OpenAlex papers cite works outside our selected subset. We keep citation edges only when both citing and cited papers are present, so both databases can represent the relationship completely.
Reconstruct abstracts	OpenAlex abstracts can be stored as an inverted index. We convert them back into readable text so text search queries are possible.

Table 2: Main ETL decisions

9.4 What “Normalize” Means Here

Normalization means splitting data into separate logical tables instead of repeating everything inside one huge table. For example, if a paper has ten authors, we do not repeat the full paper row ten times. Instead:

- the paper appears once in **papers**;
- each author appears once in **authors**;
- the connections appear in **paper_authors**.

This reduces duplication and makes relationships explicit. It also prepares the data for PostgreSQL tables and Neo4j graph relationships.

10 Step 3: Dataset Size

The current dataset contains:

Data element	Count
Papers	299
Authors	1966
Topics	1104
Paper-author relationships	2077
Paper-topic relationships	4288
Citation relationships	392

Table 3: Current dataset size

This is not the full OpenAlex database. It is a controlled subset, small enough to run locally but rich enough to compare relationships.

11 Step 4: Running the Databases with Docker

We use Docker Compose so the project can run locally in a reproducible way.

The services are:

- PostgreSQL 16;
- Neo4j 5;
- Adminer, a web interface for PostgreSQL.

Start everything with:

```
docker compose up -d
```

Check that everything is running:

```
docker compose ps
```

Stop everything with:

```
docker compose down
```

12 Step 5: PostgreSQL Implementation

PostgreSQL uses a relational model. That means data is stored in tables.

The schema is in:

```
database/postgres/schema.sql
```

The tables are:

- papers;
- authors;
- topics;
- paper_authors;
- paper_topics;
- citations.

12.1 Why These Tables Exist

Table	Purpose
papers	Stores one row per selected OpenAlex work. Important columns include title, year, DOI, abstract, citation count, and OpenAlex URL.
authors	Stores one row per author. The same author may write many papers, so we store the author once.
topics	Stores one row per OpenAlex topic. Topics are reused by many papers.
paper_authors	Bridge table connecting papers and authors. It also stores author position and corresponding-author information.
paper_topics	Bridge table connecting papers and topics. It also stores the topic score and whether the topic came from the primary topic or related topics.
citations	Stores internal citation links from one selected paper to another selected paper.

Table 4: PostgreSQL table purpose

12.2 Primary Keys, Foreign Keys, and Indexes

PostgreSQL uses primary keys to guarantee uniqueness. For example, **papers.paper_id** is the primary key of **papers**. This means the same paper cannot be inserted twice with the same ID.

PostgreSQL uses foreign keys to guarantee valid relationships. For example, **paper_authors.paper_id** references **papers.paper_id**. This prevents a relationship from pointing to a paper that does not exist.

Indexes are added on columns commonly used in filters, joins, or ordering:

- **publication_year**, useful for year filters;
- **cited_by_count**, useful for ranking papers by citation count;
- **author_id**, useful for author joins;
- **topic_id**, useful for topic joins;
- **cited_paper_id**, useful for citation lookup.

This connects to the course topic of file organization and access paths: the DBMS can use indexes to avoid scanning everything when a faster lookup is available.

12.3 How PostgreSQL Represents Relationships

PostgreSQL does not store graph edges directly. It represents relationships through rows in tables.

Example:

```
papers.paper_id
authors.author_id
paper_authors(paper_id, author_id)
```

To find which authors wrote which papers, SQL joins these tables. This is powerful and standard, but relationship-heavy queries can become long because every relationship must be reconstructed from tables.

12.4 Loading PostgreSQL

Run:

```
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/schema.sql
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/load.sql
```

Adminer is used to visually inspect PostgreSQL:

`http://localhost:8080`

Login:

```
System: PostgreSQL
Server: postgres
Username: openalex
Password: openalex
Database: openalex_ai
```

13 Step 6: Neo4j Implementation

Neo4j uses a graph model. That means data is stored as nodes and relationships.

Nodes:

- Paper;
- Author;
- Topic.

Relationships:

- (Author)-[:AUTHORED]->(Paper);
- (Paper)-[:HAS_TOPIC]->(Topic);
- (Paper)-[:CITES]->(Paper).

13.1 Why This Graph Model Fits OpenAlex

OpenAlex data is naturally graph-shaped. The main objects are papers, authors, and topics. The most interesting information is often not inside one object, but between objects:

- which author wrote which paper;
- which paper belongs to which topic;
- which paper cites another paper;
- which authors are connected through citations or shared topics.

Neo4j stores these connections directly. That makes path questions easier to express. For example, a citation network question can be written as a readable chain of nodes and relationships.

13.2 Neo4j Labels, Properties, and Constraints

In Neo4j:

- **labels** classify nodes, such as `Paper`, `Author`, and `Topic`;
- **properties** store values, such as title, display name, year, and citation count;
- **relationships** connect nodes, such as `AUTHORED`, `HAS_TOPIC`, and `CITES`.

We also define uniqueness constraints:

- every `Paper` has a unique `paper_id`;
- every `Author` has a unique `author_id`;
- every `Topic` has a unique `topic_id`.

These constraints play a similar role to primary keys in PostgreSQL. They make importing safer and make node lookup more efficient.

13.3 How Neo4j Represents Relationships

In Neo4j, the relationship is part of the stored model. For example:

```
(a:Author)-[:AUTHORED]->(p:Paper)
```

This directly means:

Author `a` wrote paper `p`.

We do not need a bridge table in Neo4j. The edge itself is the relationship. This is why Neo4j is easier to read for path-based questions.

13.4 Loading Neo4j

Run:

```
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 -f /cypher/constraints.cypher
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 -f /cypher/import.cypher
```

Neo4j Browser is available at:

<http://localhost:7474>

Login:

Username: `neo4j`
Password: `openalex123`

14 Step 7: Validating the Load

After loading both databases, we checked that the counts match.

Data element	PostgreSQL	Neo4j
Papers	299	299
Authors	1966	1966
Topics	1104	1104
Paper-author / AUTHORED	2077	2077
Paper-topic / HAS_TOPIC	4288	4288
Citations / CITES	392	392

Table 5: Validation counts

This is important because if the counts did not match, the benchmark would be unfair. The benchmark must compare the same logical dataset in both systems.

15 Step 8: Writing Equivalent Queries

We created 11 paired queries. Each query has:

- a PostgreSQL SQL version;
- a Neo4j Cypher version.

The paired workload is stored in:

`benchmarks/queries.json`

The manual SQL queries are also in:

`database/postgres/queries.sql`

The manual Cypher queries are in:

`database/neo4j/queries.cypher`

15.1 What the 11 Queries Test

The query set is designed to cover both normal analytical questions and graph-style relationship questions. This is important because a fair DBMS comparison should not use only one easy query.

Query	What it tests	Main shape
Most cited papers	Sort papers by citation count and return the top results.	Ranking

Query	What it tests	Main shape
Top authors	Count how many selected papers each author has.	Join + group
Top topics	Count how often each topic appears in the selected papers.	Join + group
Author collaboration pairs	Find authors who appear together on the same papers.	Self-join / pattern
Citation links	Return citation edges where both papers are inside our subset.	Relationship lookup
RAG-related papers	Search titles and abstracts for RAG-related terms.	Text filter
Two-hop citation paths	Find paper pairs connected through two citation steps.	Path query
Shared topics	Find author pairs connected by common topics.	Multi-join / graph pattern
Shared references	Find papers that cite the same internal references.	Self-join / graph pattern
Paths up to 3 hops	Find citation paths of length two or three.	Recursive/path query
Author citation network	Find authors connected because one author's paper cites another author's paper.	Graph traversal

15.2 What “Equivalent Query” Means

Equivalent does not mean the SQL and Cypher text look identical. They cannot look identical because the data models are different. Equivalent means:

Both queries answer the same logical question over the same data.

For example, PostgreSQL may need five joined tables while Neo4j needs one path pattern. That is acceptable because each query uses the natural language of its DBMS. The important thing is that the result meaning is the same.

15.3 Example Query

Question:

Which authors are connected because one author's paper cites another author's paper?

In Neo4j, this is naturally written as a graph path:

```
(Author)-[:AUTHORED]->(Paper)-[:CITES]->(Paper)<-[:AUTHORED)-(Author)
```

In PostgreSQL, the same idea requires joining multiple tables:

- `paper_authors;`

- citations;
- authors.

This is one of the main differences between the two systems.

16 Step 9: Benchmarking

The benchmark script is:

```
src/benchmark_queries.py
```

We run:

```
python src/benchmark_queries.py --runs 5 --warmups 1
```

Benchmark method:

- 1 warmup run per query/system;
- 5 measured runs per query/system;
- PostgreSQL measured with `EXPLAIN ANALYZE`;
- Neo4j measured with `PROFILE`;
- raw results saved as CSV.

Results are saved in:

```
benchmarks/results/benchmark_results.csv  
docs/benchmark_results.md
```

16.1 Why We Use Warmups and Averages

One query execution time can be noisy. The computer may be busy, the database may be loading metadata, or some data may already be cached in memory. To reduce random effects, the benchmark uses:

- warmup runs, which are ignored;
- multiple measured runs;
- average time for comparison.

This does not make the benchmark perfect, but it makes it more reliable than running each query once manually.

16.2 Why PostgreSQL Uses EXPLAIN ANALYZE

`EXPLAIN ANALYZE` runs the SQL query and reports the execution time and query plan. The query plan is the DBMS’s internal strategy, such as:

- scan a table;
- use an index;
- join tables;
- group rows;
- sort results.

This connects directly to the course topic of query processing. The DBMS does not simply “read SQL” line by line. It builds an execution plan and chooses physical operators.

16.3 Why Neo4j Uses PROFILE

`PROFILE` runs the Cypher query and shows the graph execution plan. It can show operations such as:

- find nodes by label or index;
- expand relationships;
- filter paths;
- aggregate results;
- sort and limit output.

This is the graph equivalent of looking at how the database executes the query internally.

16.4 What the Benchmark Does Not Prove

The benchmark is useful, but it should be interpreted carefully. It does not prove that PostgreSQL is always better than Neo4j. It proves that, for our current dataset, machine, indexes, and query set, PostgreSQL was faster on most measured queries and Neo4j was faster on the author citation network query. We later added a supplemental graph-focused workload to test more explicitly graph-shaped questions. That extra workload gives Neo4j four additional wins on multi-hop/network-style queries.

Important limitations:

- the dataset is a small OpenAlex subset, not the full OpenAlex database;
- the benchmark runs locally, not on a production server;
- both systems are containerized through Docker;
- query performance can change with more data, different indexes, or different query patterns;
- Neo4j may become more useful for deeper graph traversal and interactive network exploration.

17 Step 10: Benchmark Results

The benchmark result should be read in two layers:

- **Main workload:** PostgreSQL was faster on 10 out of 11 query pairs; Neo4j was faster on the author citation network query.
- **Supplemental graph-focused workload:** we added 9 extra query pairs to test more graph-shaped questions.
- **Combined:** PostgreSQL wins 15/20 total pairs; Neo4j wins 5 graph/network pairs.
- The point is not that the systems are tied; the point is that the winner depends on the query shape.

Table 7: Combined benchmark results: main workload plus graph-focused supplemental workload

Query	Workload	PostgreSQL ms	Neo4j ms	Faster
Most cited papers	Main	0.058	4.200	PostgreSQL
Top authors	Main	2.422	8.600	PostgreSQL
Top topics	Main	4.762	15.200	PostgreSQL
Author collaboration	Main	44.358	65.000	PostgreSQL
Citation links	Main	0.183	4.400	PostgreSQL
RAG-related papers	Main	3.515	8.600	PostgreSQL
Two-hop citation paths	Main	1.577	8.400	PostgreSQL
Shared topics	Main	309.447	586.600	PostgreSQL
Shared references	Main	2.171	7.800	PostgreSQL
Paths up to 3 hops	Main	5.104	12.400	PostgreSQL
Author citation network	Main	48.214	37.400	Neo4j
Anchored author citation neighborhood	Graph-focused	0.456	1.400	PostgreSQL
Paper citation neighborhood within two hops	Graph-focused	0.441	2.800	PostgreSQL
Paper context subgraph	Graph-focused	0.605	4.600	PostgreSQL
Topic-to-author neighborhood	Graph-focused	0.694	2.600	PostgreSQL
Topic-filtered author citation network	Graph-focused	0.563	3.800	PostgreSQL
Two-hop author citation network	Graph-focused	43.028	39.200	Neo4j
Author citation with shared paper topic	Graph-focused	783.048	338.000	Neo4j
Three-hop author citation network	Graph-focused	161.876	131.000	Neo4j
Two-hop author citation with shared topic	Graph-focused	375.604	225.400	Neo4j

17.1 Supplemental Graph-Focused Benchmark

We added a second benchmark file:

`benchmarks/graph_focused_queries.json`

The purpose is to answer a fair question:

If the original workload mostly favors PostgreSQL-style tabular analysis, what happens when we add more graph-shaped questions?

How to explain this:

- PostgreSQL still wins small anchored queries because the dataset is small and indexed lookups are very cheap.

- Neo4j wins when the query becomes more network-like, especially when it combines multi-hop citation traversal with topic overlap.
- This strengthens our conclusion: Neo4j is not automatically faster, but it becomes more competitive when the question is truly graph-shaped.

Interpretation:

- PostgreSQL is very strong for structured tables, sorting, filtering, joins, and aggregations.
- Neo4j is more natural for graph-shaped questions involving paths and relationships.
- The result depends on query type and dataset size.

17.2 How to Explain the Result Without Overclaiming

The correct conclusion is balanced:

PostgreSQL performed better for most structured analytical queries in our benchmark. Neo4j was clearer and, in one case, faster for a relationship-heavy author citation network query. Therefore, the best DBMS depends on the shape of the data and the question being asked.

Do not say:

PostgreSQL is always better.

That would be too strong and scientifically incorrect. The project shows a specific comparison under specific conditions.

17.3 Why PostgreSQL Wins Many Queries Here

Many benchmark queries are classic tabular analytics:

- sort papers by citation count;
- group papers by author;
- group papers by topic;
- filter papers by title or abstract text;
- join normalized tables and aggregate counts.

PostgreSQL is highly optimized for this type of workload. It has mature query planning, efficient joins, indexes, sorting, and aggregation operators. Our dataset is also small enough that PostgreSQL can process many queries very quickly.

17.4 Why Neo4j Still Matters

Neo4j is not included only for speed. It also matters because of expressiveness and model fit.

For graph-shaped questions, the Cypher query often looks like the real-world question. For example:

```
Author -> Paper -> Cited Paper -> Author
```

This is easier to explain visually and conceptually than several SQL joins. In the author citation network query, Neo4j was also faster in our benchmark.

17.5 What the Professor May Care About

The professor may care less about the absolute milliseconds and more about whether we understand the comparison. The important points are:

- we used the same input data for both databases;
- we modeled the same real-world entities in two different ways;
- we validated matching counts before benchmarking;
- we wrote equivalent queries;
- we interpreted results with limitations, not exaggerated claims;
- we connected the implementation to course topics.

18 Step 11: Visual Evidence

For the presentation, we collected screenshots showing:

- OpenAlex source and documentation;
- Docker containers running;
- PostgreSQL tables in Adminer;
- SQL query result;
- Neo4j database overview;
- Neo4j Author–Paper–Topic graph;
- Neo4j citation network;
- benchmark chart.

These screenshots are stored in:

`docs/figures/`

19 Step 12: Final Deliverables

The final deliverables are:

- docs/project_proposal.pdf
- docs/project_roadmap.pdf
- docs/project_report.pdf
- docs/OpenAlex_DBMS_Comparison_Sapienza.pptx
- README.md
- database scripts, benchmark scripts, query files, and results.

20 How to Explain the Project in the Presentation

Use this simple storyline:

1. We wanted to compare a relational DBMS and a graph database.
2. We chose OpenAlex because scholarly data contains papers, authors, topics, and citations.
3. We collected AI/RAG-related papers through the OpenAlex API.
4. We cleaned and normalized the data into CSV files.
5. We loaded the same CSV files into PostgreSQL and Neo4j.
6. PostgreSQL represents relationships using tables and joins.
7. Neo4j represents relationships directly as graph edges.
8. We wrote equivalent SQL and Cypher queries.
9. We benchmarked 11 query pairs.
10. PostgreSQL was faster for most structured analytical queries.
11. Neo4j was better and more natural for the author citation network query.
12. The conclusion is that the best database depends on the query shape.

21 What Leo Should Know Before Presenting

Leo does not need to memorize all code. He should understand these core points:

- The same dataset is used in both databases.
- PostgreSQL uses tables and bridge tables.
- Neo4j uses nodes and relationships.
- ETL means collecting, cleaning, and loading data.
- SQL is better for tabular aggregation.

- Cypher is clearer for path traversal.
- The benchmark result is local to our dataset and workload.
- We are not saying one database is always better than the other.

22 How This Connects to the Lecture Slides

This project is useful because it connects directly to topics from the course. We should mention these connections during the presentation because they show that the project is not only a programming exercise.

22.1 Lecture Concepts Explained Without the Slides

Graph databases. The graph database lecture explains that some data is naturally represented as a network. In a network, the connections are first-class information. Our project uses this idea because OpenAlex papers are connected to authors, topics, and other papers.

Neo4j property graph. Neo4j uses the property graph model. This means nodes and relationships can both have properties. For our project:

- **Paper** nodes have title, year, DOI, citation count, abstract, and topic properties;
- **Author** nodes have author ID and display name;
- **Topic** nodes have topic ID and display name;
- **AUTHORED** relationships can store author position and corresponding-author information;
- **HAS_TOPIC** relationships can store topic score and source;
- **CITES** relationships connect one paper to another.

Cypher pattern matching. Cypher queries use patterns. A pattern visually resembles the graph itself. For example:

```
MATCH (a:Author)-[:AUTHORED]->(p:Paper)
RETURN a.display_name, p.title
```

This reads almost like:

Find authors connected by an AUTHORED relationship to papers.

Relational joins. In relational databases, relationships are represented by values in tables. To answer relationship questions, SQL joins tables using matching IDs. For example, joining `paper_authors` with `authors` gives the names of authors for each paper.

Query processing. The query processing lectures explain that a DBMS translates a query into an execution plan. The execution plan may contain operators such as scan, selection, projection, join, grouping, sorting, and aggregation. Our benchmark connects to this because PostgreSQL uses `EXPLAIN ANALYZE` and Neo4j uses `PROFILE`.

File organization and indexes. The file organization material explains that data access is not free. The DBMS needs a way to find data efficiently. Indexes help by creating faster access paths. That is why our PostgreSQL schema defines indexes and our Neo4j schema defines uniqueness constraints.

RDF databases. RDF stores data as triples: subject, predicate, object. It is useful for semantic web, ontologies, and linked data. We did not use RDF because our approved comparison is PostgreSQL versus Neo4j property graphs.

Data warehousing. Data warehousing focuses on analytical schemas such as fact tables, dimension tables, star schemas, and OLAP. We did not build a warehouse because our project is a DBMS comparison, not a dimensional modeling project. This is important if the professor asks why there is no star schema.

Lecture topic	Connection to our project
Graph databases	Our OpenAlex data is naturally graph-shaped: authors write papers, papers have topics, and papers cite other papers.
Neo4j property graph	We use <code>Paper</code> , <code>Author</code> , and <code>Topic</code> nodes, with <code>AUTHORED</code> , <code>HAS_TOPIC</code> , and <code>CITES</code> relationships.
Cypher pattern matching	Our Neo4j queries use <code>MATCH</code> patterns, for example <code>author to paper to cited paper to author</code> .
Relational joins	PostgreSQL does not store relationships as edges, so it reconstructs them through joins and bridge tables.
Query processing	Our SQL benchmark queries use selections, projections, joins, grouping, sorting, and aggregation.
Indexes and file organization	PostgreSQL uses primary keys and indexes; Neo4j uses constraints/indexes before graph traversal.
RDF databases	We did not use RDF triples because our approved project is a property graph DBMS comparison, not a semantic web project.
Data warehousing	We did not build a star schema or OLAP project because our topic is DBMS comparison.

Table 8: Course topics connected to the project

Short explanation to say orally:

The course explains that relational databases represent relationships through joins, while graph databases represent relationships directly as edges. Our project applies this idea to OpenAlex data: in PostgreSQL we use bridge tables and joins, while in Neo4j we use graph relationships such as `AUTHORED`, `HAS_TOPIC`, and `CITES`.

23 Possible Professor Questions and Short Answers

Table 9: Possible questions and short answers

Question	Answer
Why OpenAlex?	It is open, has an API, and contains papers, authors, topics, and citations.
Why not use the full dataset?	Full OpenAlex is too large, so we used a focused AI/RAG subset suitable for local testing.
How do you know the comparison is fair?	Both databases are loaded from the same normalized CSV files and the counts match.

Question	Answer
Why is PostgreSQL faster on most queries?	Many queries are aggregations and joins on a small structured dataset, which PostgreSQL handles well.
Why use Neo4j then?	Neo4j is clearer for graph-shaped questions, paths, and network exploration.
What is the main conclusion?	PostgreSQL is strong for structured analysis; Neo4j is strong for relationship-heavy traversal.
What is ETL?	Extract data from OpenAlex, transform it into clean CSV files, then load it into PostgreSQL and Neo4j.
Why use the same CSV files for both databases?	To make the comparison fair. The input data is the same, only the database model changes.
What is a bridge table?	A relational table used to represent many-to-many relationships, such as papers connected to many authors and authors connected to many papers.
What is a graph traversal?	Following relationships from node to node, for example Author to Paper to Cited Paper to Author.
Why not RDF?	RDF is based on triples and semantic vocabularies. Our approved project compares PostgreSQL with Neo4j property graphs, not semantic web databases.
Why not data warehousing?	Data warehousing would require fact tables, dimensions, and OLAP sessions. Our project is a DBMS comparison, not a warehouse design.
What does EXPLAIN ANALYZE do?	It runs a PostgreSQL query and reports the execution plan and measured execution time.
What does Neo4j PROFILE do?	It runs a Cypher query and shows the graph query plan and execution details.
What is the biggest limitation?	The dataset is small and local, so results should not be generalized to every dataset or production environment.

24 What We Should Do Now

The project implementation is complete enough to present. The next work is not to redesign the project from zero. The next work is to prepare the explanation and make sure both group members can defend the choices.

24.1 Immediate Checklist

1. Pull the latest GitHub version so both group members have the same files.
2. Open docs/project_report.pdf and read the introduction, methodology, results, and conclusion.
3. Open docs/OpenAlex_DBMS_Comparison_Sapienza.pptx and rehearse the slide story.
4. Keep docs/leo_step_by_step_walkthrough.pdf as the explanation document for Leo.
5. Do not change benchmark numbers unless we rerun the benchmark and update the report, charts, and slides together.
6. Do not depend on live internet during the presentation; use screenshots and local files.
7. If doing a live demo, start Docker before the presentation and verify both Adminer and Neo4j Browser open correctly.

24.2 What Leo Should Focus on Learning

Leo should focus on these five ideas:

1. **Dataset story:** OpenAlex gives papers, authors, topics, and citations.
2. **ETL story:** raw API data was cleaned and normalized into CSV files.
3. **Modeling story:** PostgreSQL uses tables and joins; Neo4j uses nodes and relationships.
4. **Benchmark story:** we used equivalent SQL/Cypher queries and measured average execution time.
5. **Conclusion story:** PostgreSQL was better for most structured tabular queries in the main benchmark; Neo4j was faster for the author citation network query and four additional supplemental graph/network queries.

24.3 Suggested Presentation Split

A simple split between Hamza and Leo could be:

Slides	Speaker	Main responsibility
1–6	Hamza	Introduce the project, dataset, pipeline, ETL, and normalized dataset.
7–13	Leo	Explain PostgreSQL model, Neo4j model, screenshots, and equivalent query example.
14–18	Hamza + Leo	Explain benchmark method, results, interpretation, limitations, and final conclusion.
19–22	Leo if asked	Use appendix slides to connect the project to course concepts and the supplemental benchmark.

Table 10: Possible presentation split

This split is only a suggestion. The important thing is that both members can answer general questions.

24.4 Slide-by-Slide Speaking Notes

Slide	Title	What to say
1	Comparison between PostgreSQL and Neo4j	Introduce the topic: same OpenAlex dataset, two DBMSs, one comparison.
2	Project Goal	Explain the main question: which model is more suitable for AI research paper data, relational or graph?
3	Dataset Source: OpenAlex	OpenAlex is an open scholarly dataset with works, authors, topics, and citation references.
4	Pipeline Followed	Show the full process from API to benchmarks. This proves the project is linear and reproducible.
5	ETL Process	Explain extract, transform, load. Mention nested JSON, abstract reconstruction, deduplication, and internal citations.

Slide	Title	What to say
6	Normalized Dataset	Explain the final dataset counts. These counts show what was loaded into both DBMSs.
7	PostgreSQL Relational Model	Explain tables, primary keys, bridge tables, joins, and indexes.
8	Neo4j Graph Model	Explain nodes, labels, properties, relationships, and graph traversal.
9	Implementation Setup	Docker runs PostgreSQL, Neo4j, and Adminer locally. This makes the project reproducible.
10	PostgreSQL Evidence	Adminer screenshot proves PostgreSQL tables were loaded and can be inspected.
11	Neo4j Evidence: Author-Paper-Topic	Neo4j screenshot proves graph nodes and relationships were loaded.
12	Neo4j Evidence: Citation Network	Show why graph visualization is useful for citation relationships.
13	Equivalent Query Example	Explain that SQL and Cypher answer the same question using different models.
14	Benchmark Methodology	Explain warmups, measured runs, averages, EXPLAIN ANALYZE , and PROFILE .
15	Benchmark Results	State the main result: PostgreSQL faster on 10/11; Neo4j faster on author citation network.
16	Interpretation	Explain why: PostgreSQL fits tabular aggregation; Neo4j fits relationship traversal.
17	Limitations and Future Work	Mention small dataset, local environment, and future improvements such as larger data or deeper traversal.
18	Final Conclusions	Give the balanced final answer: database choice depends on data shape and query pattern.
19	Course Connection: Graph Databases	Use this if the professor asks about lecture relevance. Explain relationships as edges.
20	Course Connection: Joins vs Traversal	Explain that PostgreSQL reconstructs relationships through joins while Neo4j traverses stored edges.
21	Course Connection: Scope Choices	Explain why we did not present this as RDF or data warehousing. It is a DBMS comparison.
22	Supplemental Benchmark: Graph-Focused Workload	Explain that we added 9 extra graph-focused query pairs. PostgreSQL still wins small anchored lookups, but Neo4j wins four additional network-style queries.

24.5 What Not to Do at the Last Minute

- Do not regenerate the dataset with different parameters unless we are ready to reload both databases and rerun benchmarks.
- Do not change only one database model; both models must stay logically equivalent.
- Do not add new benchmark claims without measured results.
- Do not present Neo4j as universally faster or PostgreSQL as universally better.
- Do not rely on live Docker screenshots if Docker is not already running.

24.6 If the Professor Asks “What Did Leo Do?”

Leo can answer in terms of understanding and presentation ownership:

I focused on understanding the database comparison, especially the relational model versus the graph model, the equivalent SQL/Cypher queries, and how the benchmark results should be interpreted. I can explain why the same OpenAlex dataset is represented with bridge tables in PostgreSQL and with nodes and relationships in Neo4j.

If Leo wants to be more technical, he can mention:

- checking the Neo4j graph model and screenshots;
- understanding the author-paper-topic and citation network examples;
- explaining equivalent SQL and Cypher queries;
- explaining the final benchmark interpretation.

25 Commands to Reproduce the Project

From the repository root:

```
python src/collect_openalex.py --per-term 75 --per-page 75
```

```
docker compose up -d
```

```
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/schema.sql
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/load.sql
```

```
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 -f /cypher/constraints.cypher
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 -f /cypher/import.cypher
```

```
python src/benchmark_queries.py --runs 5 --warmups 1
python src/generate_benchmark_charts.py
```

26 Final Simple Conclusion

The project is complete because it includes the full pipeline:

```
OpenAlex API -> raw JSONL -> cleaned CSV -> PostgreSQL + Neo4j
-> equivalent queries -> benchmarks -> charts -> report -> presentation
```

The final message is:

PostgreSQL is better for structured tabular analysis in our current benchmark, while Neo4j is more natural for graph relationships such as citations and author networks.